

PES1UG23AM309

LAB 10

// Problem 1: Adapter Pattern

// Unified interface for flight search

interface FlightAPI {

void searchFlights(String destination);

}

// Existing Airline API classes with different methods

class AirlineA {

public void getFlightDetails(String destination) {

System.out.println("AirlineA: Getting flight details for " + destination);

}

}

class AirlineB {

public void fetchFlights(String destination) {

System.out.println("AirlineB: Fetching flights for " + destination);

}

}

class AirlineC {

public void retrieveFlightInfo(String destination) {

System.out.println("AirlineC: Retrieving flight info for " + destination);

```
}  
}
```

// Adapters to unify the interfaces

```
class AirlineAAdapter implements FlightAPI {
```

```
    private AirlineA airlineA;
```

```
    public AirlineAAdapter(AirlineA airlineA) {
```

```
        this.airlineA = airlineA;
```

```
    }
```

```
    @Override
```

```
    public void searchFlights(String destination) {
```

```
        airlineA.getFlightDetails(destination);
```

```
    }
```

```
}
```

```
class AirlineBAdapter implements FlightAPI {
```

```
    private AirlineB airlineB;
```

```
    public AirlineBAdapter(AirlineB airlineB) {
```

```
        this.airlineB = airlineB;
```

```
    }
```

```
    @Override
```

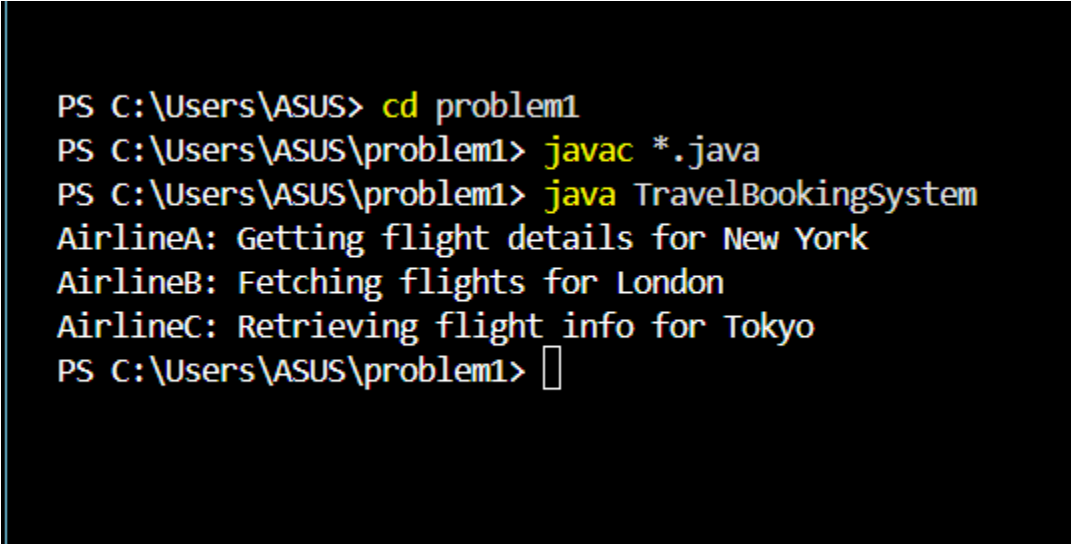
```
    public void searchFlights(String destination) {
```

```
        airlineB.fetchFlights(destination);  
    }  
}
```

```
class AirlineCAdapter implements FlightAPI {  
    private AirlineC airlineC;  
  
    public AirlineCAdapter(AirlineC airlineC) {  
        this.airlineC = airlineC;  
    }  
  
    @Override  
    public void searchFlights(String destination) {  
        airlineC.retrieveFlightInfo(destination);  
    }  
}
```

```
// Client code  
  
public class TravelBookingSystem {  
    public static void main(String[] args) {  
        FlightAPI api1 = new AirlineAAdapter(new AirlineA());  
        FlightAPI api2 = new AirlineBAdapter(new AirlineB());  
        FlightAPI api3 = new AirlineCAdapter(new AirlineC());  
  
        api1.searchFlights("New York");  
        api2.searchFlights("London");  
    }  
}
```

```
        api3.searchFlights("Tokyo");  
    }  
}
```



```
PS C:\Users\ASUS> cd problem1  
PS C:\Users\ASUS\problem1> javac *.java  
PS C:\Users\ASUS\problem1> java TravelBookingSystem  
AirlineA: Getting flight details for New York  
AirlineB: Fetching flights for London  
AirlineC: Retrieving flight info for Tokyo  
PS C:\Users\ASUS\problem1> 
```

PROBLEM 2:

// Problem 2: Proxy Pattern

// Image interface

```
interface Image {  
    void loadImage();  
    void displayImage();  
}
```

// Real Image class that performs heavy loading

```
class ReallImage implements Image {
```

```
private String filename;
```

```
public ReallImage(String filename) {  
    this.filename = filename;  
    loadImage();  
}
```

```
@Override
```

```
public void loadImage() {  
    System.out.println("Loading image: " + filename + " (heavy operation)");  
}
```

```
@Override
```

```
public void displayImage() {  
    System.out.println("Displaying image: " + filename);  
}  
}
```

```
// Proxy Image class that lazy loads and caches
```

```
class ProxyImage implements Image {  
    private ReallImage reallImage;  
    private String filename;  
    private static java.util.Map<String, ReallImage> cache = new java.util.HashMap<>();  
  
    public ProxyImage(String filename) {  
        this.filename = filename;
```

```
}
```

```
@Override
```

```
public void loadImage() {
```

```
    if (reallImage == null) {
```

```
        if (cache.containsKey(filename)) {
```

```
            reallImage = cache.get(filename);
```

```
            System.out.println("Image loaded from cache: " + filename);
```

```
        } else {
```

```
            reallImage = new ReallImage(filename);
```

```
            cache.put(filename, reallImage);
```

```
        }
```

```
    }
```

```
}
```

```
@Override
```

```
public void displayImage() {
```

```
    loadImage(); // Lazy loading
```

```
    reallImage.displayImage();
```

```
}
```

```
}
```

```
// Client code
```

```
public class ImageGalleryApp {
```

```
    public static void main(String[] args) {
```

```
        Image image1 = new ProxyImage("image1.jpg");
```

```
Image image2 = new ProxyImage("image2.jpg");
```

```
// First display - loads
```

```
image1.displayImage();
```

```
image2.displayImage();
```

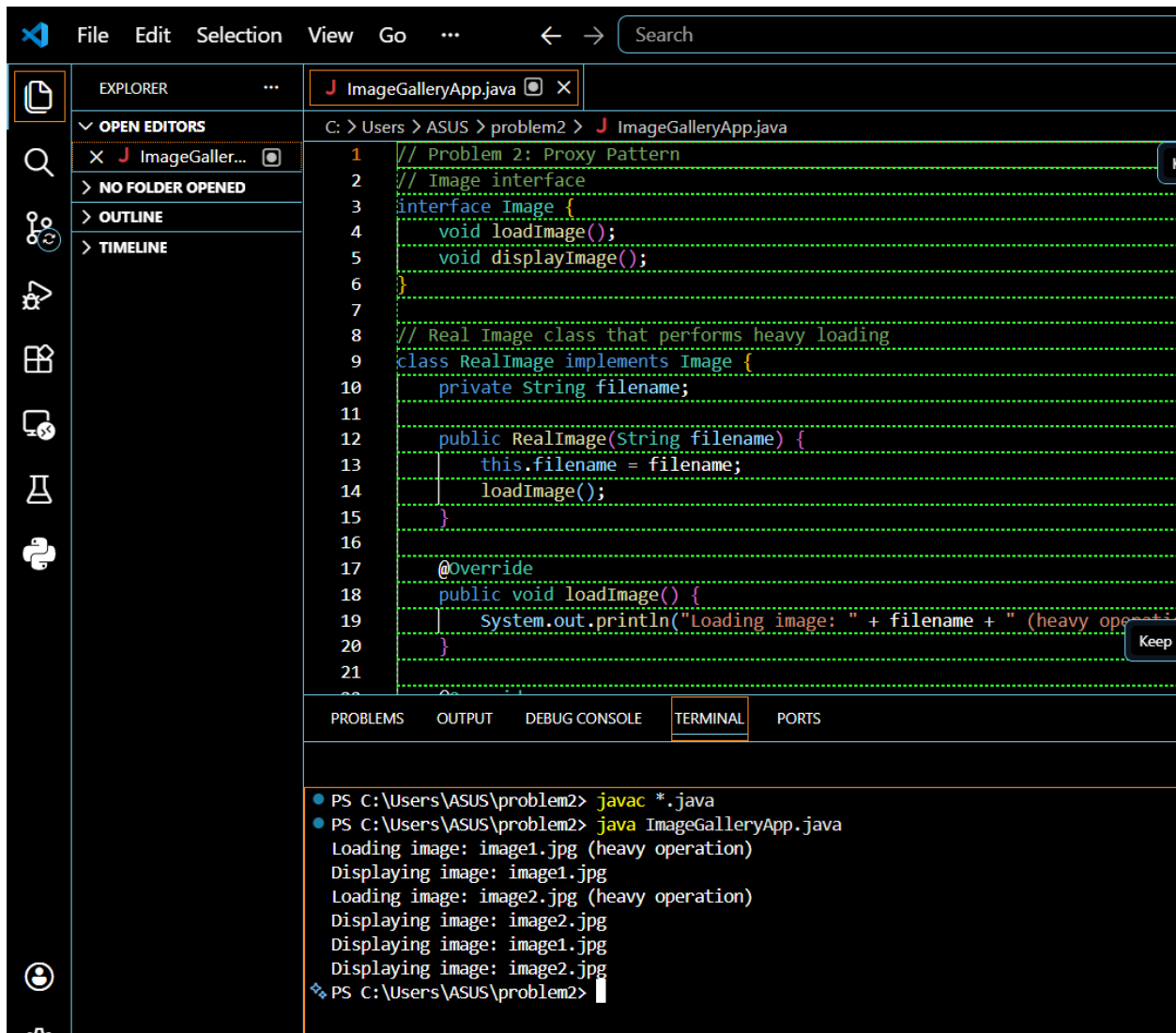
```
// Second display - uses cache
```

```
image1.displayImage();
```

```
image2.displayImage();
```

```
}
```

```
}
```



Problem 3:

// Problem 3: Flyweight Pattern

import java.util.HashMap;

import java.util.Map;

// Flyweight interface

interface Marker {


```
    void display(int x, int y);  
}
```

```
// Concrete Flyweight: shared intrinsic state
```

```
class ConcreteMarker implements Marker {
```

```
    private String icon;
```

```
    private String color;
```

```
    public ConcreteMarker(String icon, String color) {
```

```
        this.icon = icon;
```

```
        this.color = color;
```

```
    }
```

```
    @Override
```

```
    public void display(int x, int y) {
```

```
        System.out.println("Displaying marker with icon: " + icon + ", color: " + color + " at (" + x + ",  
" + y + ")");
```

```
    }
```

```
}
```

```
// Flyweight Factory
```

```
class MarkerFactory {
```

```
    private static Map<String, Marker> markers = new HashMap<>();
```

```
    public static Marker getMarker(String icon, String color) {
```

```
        String key = icon + "_" + color;
```

```
    if (!markers.containsKey(key)) {  
        markers.put(key, new ConcreteMarker(icon, color));  
        System.out.println("Creating new marker: " + key);  
    } else {  
        System.out.println("Reusing existing marker: " + key);  
    }  
    return markers.get(key);  
}  
}
```

// Client class that holds extrinsic state

```
class MapMarker {  
    private Marker flyweight;  
    private int x, y;  
  
    public MapMarker(String icon, String color, int x, int y) {  
        this.flyweight = MarkerFactory.getMarker(icon, color);  
        this.x = x;  
        this.y = y;  
    }  
  
    public void display() {  
        flyweight.display(x, y);  
    }  
}
```

```
// Client code
```

```
public class MapApplication {  
    public static void main(String[] args) {  
        MapMarker marker1 = new MapMarker("restaurant", "red", 10, 20);  
        MapMarker marker2 = new MapMarker("hospital", "blue", 30, 40);  
        MapMarker marker3 = new MapMarker("restaurant", "red", 50, 60); // Same intrinsic state  
  
        marker1.display();  
        marker2.display();  
        marker3.display();  
    }  
}
```

PROBLEM 4:

```
java// Problem 4: Facade Pattern
```

```
// Subsystem classes for university portal services
```

```
class CourseRegistrationService {  
    public void registerCourse(String courseName) {  
        System.out.println("Registering course: " + courseName);  
    }  
}
```

```
class FeePaymentService {  
    public void payFees(double amount) {  
        System.out.println("Paying fees: $" + amount);  
    }  
}
```

```
}  
}
```

```
class ResultService {  
    public void viewResults(String studentId) {  
        System.out.println("Viewing results for student ID: " + studentId);  
    }  
}
```

```
class CertificateService {  
    public void applyCertificate(String certificateType) {  
        System.out.println("Applying for certificate: " + certificateType);  
    }  
}
```

// Facade that simplifies student portal access

```
class StudentPortalFacade {  
    private CourseRegistrationService registrationService;  
    private FeePaymentService feePaymentService;  
    private ResultService resultService;  
    private CertificateService certificateService;  
  
    public StudentPortalFacade() {  
        this.registrationService = new CourseRegistrationService();  
        this.feePaymentService = new FeePaymentService();  
        this.resultService = new ResultService();  
    }  
}
```

```

        this.certificateService = new CertificateService();
    }

    public void studentPortalAccess(String studentId, String courseName, double feeAmount,
String certificateType) {
        System.out.println("--- Student Portal Access ---");
        registrationService.registerCourse(courseName);
        feePaymentService.payFees(feeAmount);
        resultService.viewResults(studentId);
        certificateService.applyCertificate(certificateType);
        System.out.println("--- End of portal interaction ---");
    }
}

// Client code
public class StudentPortalClient {
    public static void main(String[] args) {
        StudentPortalFacade portal = new StudentPortalFacade();
        portal.studentPortalAccess("S12345", "Data Structures", 1500.00, "Graduation Certificate");
    }
}

```

File Edit Selection View Go ...

← → Search

EXPLORER

ImageGalleryApp.java MapApplication.java StudentPortalClient.java

OPEN EDITORS

ImageGaller... MapApplicat... StudentPorta...

NO FOLDER OPENED

OUTLINE

TIMELINE

C: > Users > ASUS > problem4 > StudentPortalClient.java

1 java// Problem 4: Facade Pattern
2 // Subsystem classes for university portal services
3 class CourseRegistrationService {
4 public void registerCourse(String courseName) {
5 System.out.println("Registering course: " + courseName);
6 }
7 }
8
9 class FeePaymentService {
10 public void payFees(double amount) {
11 System.out.println("Paying fees: \$" + amount);
12 }
13 }
14
15 class ResultService {
16 public void viewResults(String studentId) {
17 System.out.println("Viewing results for student ID: " + studentId);
18 }
19 }
20

Keep Undo 5 of 5

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ASUS\problem3> java MapApplication.java
PS C:\Users\ASUS\problem3> cd ..
PS C:\Users\ASUS> cd problem4
PS C:\Users\ASUS\problem4> javac *.java
PS C:\Users\ASUS\problem4> java StudentPortalClient
--- Student Portal Access ---
Registering course: Data Structures
Paying fees: \$1500.0
Viewing results for student ID: S12345
Applying for certificate: Graduation Certificate
--- End of portal interaction ---
PS C:\Users\ASUS\problem4>

File

Edit

Selection

View

Go

...

←

→

Search

EXPLORER

...

J ImageGalleryApp.java

J MapApplication.java

OPEN EDITORS

J ImageGaller...

J MapApplicat...

NO FOLDER OPENED

OUTLINE

TIMELINE

C: > Users > ASUS > problem3 > J MapApplication.java

1

// Problem 3: Flyweight Pattern

2

import java.util.HashMap;

3

import java.util.Map;

4

5

// Flyweight interface

6

interface Marker {

7

void display(int x, int y);

8

}

9

10

// Concrete Flyweight: shared intrinsic state

11

class ConcreteMarker implements Marker {

12

private String icon;

13

private String color;

14

15

public ConcreteMarker(String icon, String color) {

16

this.icon = icon;

17

this.color = color;

18

}

19

20

@Override

21

public void display(int x, int y) {

22

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\ASUS\problem3> javac *.java

PS C:\Users\ASUS\problem3> java MapApplication.java

Creating new marker: restaurant_red

Creating new marker: hospital_blue

Reusing existing marker: restaurant_red

Displaying marker with icon: restaurant, color: red at (10, 20)

Displaying marker with icon: hospital, color: blue at (30, 40)

Displaying marker with icon: restaurant, color: red at (50, 60)

PS C:\Users\ASUS\problem3>